

ContextRay

Deterministic Context Optimization for LLM Chat Histories

A technical white paper covering the concept, architecture, API contract, empirical findings, and the known problems and limitations of ContextRay v0.3.0.

Property	Value
Version	0.3.0 (structural segmentation layer)
Package	contextray — pip install contextray
Repository	github.com/RudraMistry-cmd/contextray
License	MIT
Python support	≥ 3.9, standard library only (zero runtime dependencies)
Test status	222 checks — 18 API + 181 pipeline + 23 segmentation, all passing

Contents

1 The Concept	3
1.1 Why this problem matters	3
1.2 What it deliberately is NOT	3
1.3 Design principles	3
2 Architecture	4
2.1 Stage 0 — Structural Segmentation (segmentation.py)	4
2.2 Stage 1 — Chunking (chunking.py)	5
2.3 Stage 2 — Duplicate detection (detection.py)	5
2.4 Stage 3 — Optimization (optimization.py)	5
2.5 Stage 4 — Reporting (reporting.py)	5
3 Public API Contract	6
3.1 Python API	6
metrics schema	6
3.2 CLI	6
Terminal report layout	7
4 Input Validation & Error Handling	8
5 Determinism & Verification	9
6 Empirical Findings (A/B with llama3.2:3b)	10
Finding 1 — Naturally written LLM dialogue contains ~0% exact duplicates	10
Finding 2 — The marker text can bias models (documented risk)	10
Finding 3 — Guidance derived from the test	10
7 Safety Guarantees	11
8 Known Problems & Limitations	12
9 Integration Guide	13
Multi-turn chat loop (the standard use case)	13
Raw text and documents	13
Agent / multi-agent flows	13
Structured data inside messages	13
Golden rule	13
10 Appendix A — Worked Example	14
Realistic scenario (development fixture)	14
11 Appendix B — Quick Reference	15
Segmentation decisions at a glance	15
Pipeline decisions at a glance	15
Constants	15
CLI facts	15
Project facts	15

1 The Concept

ContextRay is a deterministic, character-level optimizer for LLM chat histories. It takes the exact message list an application would send to a model — a sequence of `{'role': ..., 'content': ...}` dictionaries — and returns a drop-in replacement list with byte-identical duplicates removed, plus precise impact metrics and a human-readable report. Nothing is summarized, paraphrased, or inferred; every transformation is a function of the input bytes alone.

The core insight is that in production LLM systems, most context bloat does not come from verbose prose. It comes from the runtime: frameworks that re-inject state, frameworks that replay completed turns, cache/retry mechanisms that append the same assistant payload twice, and agents that hand the same transcript forward at every step. This redundant traffic is byte-identical, and byte-identical content can be removed with zero information loss — deterministically and verifiably.

1.1 Why this problem matters

- **Cost.** Tokens are billed per character. Repeating content in every round multiplies cost linearly with conversation length.
- **Latency.** Model response time grows with context length; trimmed contexts respond faster.
- **Reasoning quality.** Long contexts degrade retrieval accuracy — the needle-in-a-haystack problem. Noise dilutes the signal a model attends to.
- **Debugging and auditability.** A byte-safe, deterministic transform can be reproduced offline, tested, and reasoned about; an ML summarizer cannot.

1.2 What it deliberately is NOT

ContextRay does not compress with embeddings, does not summarize, does not use a language model at any point, and cannot hallucinate. It operates on exact string equality only. This is a deliberate philosophical choice: the tool's guarantees are only as strong as the deterministic laws it is built on, and those laws hold everywhere, on every input, in every environment.

1.3 Design principles

- **Byte-safety first.** Removal only. Every transformation either keeps text verbatim or deletes an exact duplicate of it. No rewrite ever happens.
- **Deterministic everything.** Same input, same output, same hashes, same report — across runs, machines, and platforms.
- **Structural protection.** Code, JSON, and fenced blocks are recognized structurally (fences, parsers) and masked before the pipeline runs, so they can never be split or corrupted internally (Section 2.1).
- **Fail fast, fail safe.** Invalid input raises an error naming the offending message and field; if classification work itself fails, the pipeline degrades to a documented plain-text fallback instead of crashing (Section 4).
- **Conservative removal.** Only same-speaker byte-repeats are removed; everything else is reported, not touched.

2 Architecture

The package is a segmentation layer plus a four-stage core. Every stage lives in its own module with a single public function, so the pipeline can be inspected, tested, and reused stage by stage.

segment_text (segmentation.py)	chunk_and_hash (chunking.py)	detect_duplicates (detection.py)	optimize_chunks (optimization.py)	generate_metrics_and_report (reporting.py)
-----------------------------------	---------------------------------	-------------------------------------	--------------------------------------	---

```

raw text / messages
|
v
[0] split into STRICT / FLEXIBLE segments; mask STRICT with length-preserving
    placeholders (fenced code, whole-line JSON, parseable bare Python stay
    byte-exact even if duplicated)
|
v
[1] split into chunks <= 1000 chars, sha256-hash every chunk >= 64 chars
|
v
[2] mark each chunk KEPT / REMOVED / FLAGGED_ONLY using global + per-role
    lookup tables
|
v
[3] replace REMOVED chunks with a short marker (guarded against negative
    reduction)
|
v
[4] restore placeholders byte-for-byte; compute metrics, top waste blocks,
    per-role stats, human-readable report

```

The orchestrators `optimize_context()` and `optimize_text()` in `core.py` wire the stages together and are the only documented public entry points. Their return contracts are fixed (Section 3); the internal chunk representation never leaks into them.

2.1 Stage 0 — Structural Segmentation (segmentation.py)

Before deduplication runs, `segment_text()` splits the input into ordered, contiguous, byte-exact segments. Each segment is either **STRICT** (provably structural — masked out of the pipeline with a placeholder, restored byte-for-byte afterwards) or **FLEXIBLE** (deduplicated like ordinary text). The rules are purely structural — no NLP, no guessing:

Type	Detection rule	Mode
code	```-fenced block with a language tag on the opening fence	STRICT
block	```-fenced block with no language tag	STRICT
json	one or more whole lines that parse as a single JSON value (verified with <code>json.loads</code> ; fences take precedence)	STRICT
code	at least 3 whole lines that parse as valid Python (verified with <code>ast.parse</code>) — covers code pasted without fences	STRICT
text	everything else	FLEXIBLE

Masking replaces each **STRICT** segment with a “`__SEG...`” token padded with an invisible filler to the segment's exact original length, so the downstream chunking behavior and metrics are unchanged (masked length == original length). Byte-identical **STRICT** segments share one token, so duplicates among protected regions collapse like any other duplicate — and the reference copy is restored byte-for-byte.

The unfenced-Python scan restarts from every candidate line, which is superlinear on fence-free prose. A firewall caps the cost: inputs above 20,000 lines (`max_lines_for_python_scan=20000`) skip that scan entirely — fence and JSON detection, which are cheap and linear, still run. Pass `None` to scan at any size.

If segmentation itself raises for any reason, `optimize_text()` falls back to running the pipeline over the raw text (safe, but without **STRICT** protection): `metrics["segmentation_fallback"]` is `True` and the report is prefixed with a note.

Errors raised by the pipeline itself are never swallowed.

2.2 Stage 1 — Chunking (chunking.py)

Messages are split into chunks with a hard ceiling of 1,000 characters (MAX_CHUNK_SIZE). Splitting prefers natural boundaries in this order: double-newline paragraph breaks, then single-newline line breaks, then the nearest whitespace. Delimiters are kept attached so text is never deleted.

Fenced code blocks (triple backticks) are masked with placeholders before splitting, so a fence is never cut in half; the real text is restored after chunking. Chunks shorter than 64 characters (MIN_CHUNK_SIZE) are treated as noise: they are not hashed (hash = None) and are deduplicated by their raw text instead. Every other chunk receives sha256(text.strip()).

Constant	Value	Effect
MAX_CHUNK_SIZE	1000	split ceiling: paragraphs → lines → whitespace
MIN_CHUNK_SIZE	64	below this: hash = None, never replaced
CODE_FENCE_REGEX	```.*?``` (DOTALL)	placeholder-masked before splitting
DEDUP_KEY	hash or raw text	tiny chunks deduplicate by raw text

2.3 Stage 2 — Duplicate detection (detection.py)

Two lookup tables track what has been seen: a global one and a per-role one. Each chunk is classified:

Situation	Action	Rationale
role == 'system'	KEPT	System prompts are instructions; never touched.
Same key, same role (repeated)	REMOVED	Same speaker repeated itself verbatim — safe to elide.
Same key, different role	FLAGGED_ONLY	Reported as waste, but kept: removing could drop content one speaker still needs.
First occurrence	KEPT	Becomes the reference copy; records its id for the marker.

The removal rule is deliberately asymmetric: a user message is never deleted because an assistant once wrote identical text. Only same-speaker byte-repeats are removed.

2.4 Stage 3 — Optimization (optimization.py)

Removed chunks are replaced with a 33-character marker [duplicate of chunk #N removed], where N is the chunk id of the reference copy — useful for debugging and auditing. A guard prevents negative reduction: if the marker would be as long as or longer than the chunk it replaces, the chunk is kept verbatim instead. Output characters therefore never exceed input characters.

Marker guard is the reason small chunks are never replaced: for a 60-character chunk the marker (33 chars) would fit, but for chunks below 64 chars the pipeline deliberately stays conservative and skips replacement entirely.

2.5 Stage 4 — Reporting (reporting.py)

The final stage computes impact metrics and a human-readable report. Waste per block is (count – 1) × chunk_length; the worst five blocks are reported, sorted by chars wasted. Per-role redundancy percentages are computed and sorted worst-first. Safety notes are always appended, so every report documents what was NOT touched and why. A token note states which estimate was used — the default English heuristic (chars ÷ 4) or a caller-provided token_estimator. In the optimize_text() path a “Segments:” section is inserted into the report listing each segment type with its count and mode (protected vs processed).

3 Public API Contract

3.1 Python API

Two entry points cover both input shapes. `optimize_context()` takes the exact message list an application would send to a model; `optimize_text()` takes a plain string and pre-processes it with the segmentation layer first.

```
from contextray import optimize_context, optimize_text

result = optimize_context(messages)      # messages: [{'role', 'content'}, ...]
result = optimize_text(raw_text)        # raw_text: any str

result["optimized_context"]             # drop-in message list (role/content dicts)
result["metrics"]                       # impact numbers
result["top_waste_blocks"]              # worst 5 duplicate blocks
result["report"]                       # full human-readable summary
result.get("segments")                 # per-type segmentation stats (text path only)
```

The signatures `optimize_context(messages, *, config=None, token_estimator=None, **kwargs)` and `optimize_text(text, *, config=None, token_estimator=None, **kwargs)` are stable. `config` is accepted but currently ignored — it is reserved for future tuning knobs, and callers are not broken by its presence. A plain str passed to `optimize_context()` is treated as a single “text”-role message.

Return key	Type	Meaning
optimized_context	list[dict]	Reconstructed messages, {'role': str, 'content': str} only — no chunk internals.
metrics	dict	Character and token impact numbers (below).
top_waste_blocks	list[dict]	Worst duplicate blocks, max 5, sorted by chars_wasted desc.
report	str	Full printable summary incl. per-role stats and safety notes.
segments	dict	<code>optimize_text()</code> only: per-type counts and modes (code/json/block/text), e.g. {"code": {"count": 4, "mode": "protected"}}.

metrics schema

```
{
  "total_chars_in": 3566,
  "total_chars_out": 2759,
  "chars_saved": 807,
  "reduction_percentage": 22.6,
  "est_tokens_in": 891.5,
  "est_tokens_saved": 201.75,
  "segmentation_fallback": false,  # optimize_text() path
}
```

Token estimates use the English heuristic $\text{characters} \div 4$ — approximate by design (Section 8). Both functions accept an optional `token_estimator: Callable[[str], float]` keyword that replaces the heuristic: it is called with the full original text and the full optimized text, `est_tokens_saved = est_tokens_in - est_tokens_out`, and errors are re-raised with context (“token_estimator raised on optimized text: ...”).

Each `top_waste_blocks` entry is `{'hash', 'role', 'count', 'chars_wasted'}`. Per-role redundancy stats exist but are exposed only inside the report string.

3.2 CLI

```
usage: contextray optimize [-h] [--output OUTPUT] [--stdout] [--strict]
                           [--max-input-mb MAX_INPUT_MB]
                           input

positional arguments:
  input                Path to a JSON message list, or any text file.

options:
  -h, --help            show this help message and exit
  --output, -o OUTPUT  Where to write the optimized JSON
                       (default: <input>_optimized<ext>)
  --stdout              Print the optimized JSON to stdout instead of a file
  --strict              Require the exact JSON message-list format; no
                       text auto-detection
  --max-input-mb FLOAT Reject input files larger than this many MB
                       (default: 50)
```

The CLI auto-detects the input: a valid JSON message list is optimized as structured chat via `optimize_context()`; any other text (a .txt/.md file, logs, a raw dump) is optimized as a single “text” message via `optimize_text()`, which runs the segmentation layer and prints a SEGMENTS section. Pass `--strict` to require the exact JSON message-list format.

The CLI writes three of the five result keys to the output file (`optimized_context`, `metrics`, `top_waste_blocks`); the printable report is rendered on the console. Exit codes: 0 success, 1 error, 2 usage error. Validation errors are reported to `stderr` with an `[ERROR]` / `■` marker and never raise a traceback.

Terminal report layout

```
=== CONTEXTRAY OPTIMIZATION REPORT ===
SEGMENTS                                (text inputs only)
- code: 4 (protected)
- json: 1 (protected)
- block: 1 (protected)
- text: 6 (processed)

IMPACT
Original: 3029 chars
Optimized: 1624 chars
Saved: 1405 chars (46%)
TOP WASTE
- Hash 300b1660169c... repeated 2 times (990 chars wasted)
- Hash b8a2b3980551... repeated 2 times (450 chars wasted)
SAFETY
[OK] System messages preserved
[OK] Cross-role duplicates not removed
[OK] Code blocks protected
[OK] Small chunks skipped
-----
Output saved to: input_optimized.json
```

On Unicode terminals the section labels and markers render with glyphs (■■ IMPACT, ■■ TOP WASTE, ■■■ SAFETY, ✓, ■); on legacy code pages (e.g. cp1252) the CLI falls back to ASCII automatically. In `--stdout` mode the payload goes to `stdout` and the report goes to `stderr`, so shell redirection stays clean (`2>/dev/null` discards the report).

4 Input Validation & Error Handling

Both APIs validate input strictly, in order, before any processing starts. The Python API fails fast with `InvalidMessageError` (a `ValueError` subclass) that names the offending message index and the exact problem — no coercion happens (None is not auto-converted to "", block lists are not stringified):

Input problem	Error (message index i)
'content' is None (tool-call-only turns)	<code>InvalidMessageError: message[i]: 'content' is None, expected str</code>
'content' is a list of typed parts (image/text arrays)	<code>InvalidMessageError: message[i]: 'content' is a list, expected str</code>
missing 'role' or 'content' key	<code>InvalidMessageError: message[i]: missing required key 'role' / 'content'</code>
item is not a dict	<code>InvalidMessageError: message[i]: expected a dict with 'role' and 'content'</code>
'role' or 'content' not a string	<code>InvalidMessageError: message[i]: 'role' and 'content' must be strings</code>

Validate your data at the boundary: a plain str is accepted (treated as a single “text” message), dicts with extra keys are fine (extra keys are ignored), and everything else raises.

The CLI applies the same checks with JSON-friendly wording, in order:

- The file must exist and be readable, else: `could not read <path>`.
- The file size must not exceed `--max-input-mb` (default 50.0): `“input file is 61.2MB, exceeds --max-input-mb limit of 50.0MB (raise the limit or split the file)”`.
- The file must be valid JSON, else the canonical message `“Invalid input format. Expected: [{‘role’: ‘...’, ‘content’: ‘...’}]”` with a reason.
- The JSON must be a list at the top level.
- Every item must be a dict with string 'role' and 'content' keys.

The entire CLI entry point is wrapped in a catch-all handler, so even unexpected runtime errors print `“Error: <message>”` and exit 1 instead of a traceback.

5 Determinism & Verification

Every transformation is deterministic: sha256 hashing, ordered table lookups, marker generation, and report formatting are functions of the input bytes alone. The verification suite runs without any external dependencies (plain assert-based scripts, no pytest required):

Suite	Checks	Covers
tests/test_core.py	18	Public API contract: keys, message shape, determinism, no chunk leakage, InvalidMessageError cases, token_estimator (default, custom, error wrapping).
tests/test_pipeline.py	181	Stage invariants, chunk sizes, role rules, unicode, code fences, 60k+ char floods, MEGA scenarios, CLI text/json modes, SEGMENTS section, --max-input-mb, optional real-Ollama contexts.
tests/test_segmentation.py	23	Segments contiguous and byte-exact, fence/JSON/bare-Python rules, strict bytes survive optimization, metrics exact with masking, duplicate STRICT segments collapse, precedence rules, python-scan line guard, fallback, segments breakdown.

222 checks in total, all passing. The pipeline suite writes its stress fixtures to test_contexts.txt (repo root) so every run is inspectable. A local Ollama is used when present and skipped with --skip-ollama in offline mode — the deterministic scenarios never require it.

6 Empirical Findings (A/B with llama3.2:3b)

A controlled study was run against a local llama3.2:3b at temperature 0 to measure where ContextRay actually pays off — and where it introduces risk. Three findings shaped the design:

Finding 1 — Naturally written LLM dialogue contains ~0% exact duplicates

Models paraphrase; byte-identical repeats are rare in natural dialogue. The realistic redundancy comes from the runtime — re-injected state, replayed turns, doubled cache/retry payloads. In one measured double-registered-turn scenario ContextRay achieved an 11.7% reduction on the later call. Expect ~0% savings on genuinely conversational transcripts (Section 8).

Finding 2 — The marker text can bias models (documented risk)

Transcript variant	Reviewer verdict (5 runs)	Determinism
Full duplicate present	REJECTED 5/5	Yes — temp 0, identical prompt
[duplicate of chunk #4 removed] marker	APPROVED 5/5	Yes — temp 0, identical prompt
Duplicate silently removed	REJECTED 5/5	Yes — temp 0, identical prompt

The model read the marker as “cleanup already performed — code quality improved” and drifted toward approval. The removal itself is safe: silent removal scored identically to the full text. The marker string is the risk, not the deletion.

Finding 3 — Guidance derived from the test

- **Non-critical logging / archival:** the default marker is fine.
- **Agent pipelines** where a downstream agent makes go/no-go decisions on the transcript: consider a neutral elision wording or a silent-removal option, and re-test with the target model family.

This is a behavior-observation document, not a guarantee. Marker bias varies by model family and prompt; the package ships the informative marker by default and documents the trade-off.

7 Safety Guarantees

- **System messages preserved.** The 'system' role is always KEPT — never touched by the pipeline.
- **Cross-role duplicates are never removed.** Same text in different roles is flagged and reported, never deleted.
- **STRICT segments survive byte-exact.** Fenced code, fenced blocks, whole-line JSON, and parseable bare Python are masked before deduplication and restored byte-for-byte afterwards — even when duplicated, only the first occurrence is kept and it is untouched.
- **No negative reductions.** A chunk is replaced by a marker only when the marker is strictly smaller; output characters never exceed input characters.
- **Tiny chunks are conservative.** Chunks below 64 chars are reported as waste but never replaced.
- **Placeholders never leak.** Masked text always restores to the original bytes; the placeholder prefix is chosen collision-free per run.
- **Guarded worst-case costs.** The unfenced-Python scan is skipped above 20,000 lines, and the CLI rejects oversized inputs (`--max-input-mb`, default 50 MB) instead of reading them.
- **Safe fallback.** If segmentation itself fails, `optimize_text()` runs the pipeline over the raw text and reports `segmentation_fallback=True` — the caller is never left without a result.

8 Known Problems & Limitations

This section is written to be read. ContextRay makes explicit trade-offs; each is listed with its impact and the mitigation that exists today.

Limitation	Impact	Mitigation / status
Exact byte-level duplicates only	Semantic repeats, paraphrases, and near-duplicates are never removed — savings ceiling is low on natural dialogue.	Documented expectation (~0% on conversational text); runtime redundancy is the target use case.
Marker string can bias LLMs	A downstream model may read '[duplicate of chunk #N removed]' as a quality signal and drift its verdict.	Empirically documented (Section 6); neutral wording or silent removal advised for decision pipelines.
Structural coverage is finite	Only fenced blocks, whole-line JSON, and parseable bare Python (≥ 3 lines) are recognized; inline code (<code>`x = 1`</code>) and malformed fences chunk like plain text.	Recognition is parser-verified (<code>json.loads / ast.parse</code>) — no guessing. Unrecognized regions are never corrupted, worst case they deduplicate like text.
Whitespace-sensitive hashing	<code>text.strip()</code> folds outer whitespace ('hello' == 'hello\n') but inner spacing stays significant ('a b' != 'a b').	Conservative direction — false negatives over false positives.
Tiny chunks: flagged but never removable	Duplicates under 64 chars appear in top-waste reporting but are never replaced.	Visibility without action; the marker-length guard makes removal unsafe for micro-chunks.
Cross-role duplicates never removed	Same text in user and assistant turns stays in full — leaves savings on the table.	Deliberate safety choice; reported as waste so users can decide manually.
Token estimates are a heuristic	<code>chars ÷ 4</code> is an English average; real tokenizer counts differ, especially for code and non-English text.	Stated as estimates in the schema and report; a custom <code>token_estimator</code> callable replaces the heuristic everywhere.
Unfenced-Python scan is capped	Inputs above 20,000 lines skip bare-Python detection, so large fence-free documents lose that protection.	Fences and JSON still detected (linear cost); the threshold is configurable (<code>max_lines_for_python_scan</code> , <code>None</code> = unlimited).
Fallback loses STRICT protection	If segmentation raises, <code>optimize_text()</code> runs unprotected over the raw text.	Documented and flagged (<code>segmentation_fallback=True</code> + report prefix); output is still valid, just not protected.
CLI rejects oversized files	Inputs above <code>--max-input-mb</code> (default 50 MB) are refused.	Raise the limit or split the file; the message says both.
config kwarg ignored	Chunk size limits and thresholds are not yet tunable at runtime.	Accepted for forward compatibility; module constants are the current knobs.

Open questions for the next releases:

- Should role stats, chunk sizes, and skipped-chunk counts enter the machine-readable contract?
- Should the marker text be configurable (a `--no-markers` / custom-wording option)?
- Should MIN/MAX chunk sizes become real config knobs now that the fallback path is exercised?

9 Integration Guide

Multi-turn chat loop (the standard use case)

```
from contextray import optimize_context

history = [{"role": "system", "content": SYSTEM_PROMPT}]
while True:
    user_in = input("You: ")
    history.append({"role": "user", "content": user_in})
    history = optimize_context(history)["optimized_context"]
    reply = call_model(history)
    history.append({"role": "assistant", "content": reply})
```

Optimizing every round keeps growth logarithmic: only genuinely new content is added while everything repeated is collapsed.

Raw text and documents

```
from contextray import optimize_text

notes = open("notes.md", encoding="utf-8").read()
result = optimize_text(notes) # segmentation runs first
print(result["segments"])    # {'code': {'count': 4, 'mode': 'protected'}, ...}
optimized = "\n".join(m["content"] for m in result["optimized_context"])
```

Fenced code, JSON dumps, and pasted Python in the document are masked, deduplicated (if repeated), and restored byte-for-byte; the surrounding prose deduplicates like any chat message. The CLI takes the same path automatically for non-JSON files.

Agent / multi-agent flows

```
def agent_turn(messages, handoff):
    to_send = messages + [{"role": "user", "content": handoff}]
    optimized = optimize_context(to_send)["optimized_context"]
    out = call_model(optimized)
    messages.append({"role": "assistant", "content": out})
    return out
```

Structured data inside messages

JSON content is plain text: the first occurrence is kept verbatim; an exactly repeated block is collapsed like any duplicate. Downstream parsers keep working as long as they read the first occurrence, which is untouched.

Golden rule

Whatever list you would send to the model is what you feed to `optimize_context()`; whatever raw text you would paste is what you feed to `optimize_text()`. The output is a drop-in replacement — role and content strings only, in the original order. OpenAI, Ollama HTTP, and streaming payloads are all already compatible; streaming requires only that the caller accumulates the content parts into a string first.

10 Appendix A — Worked Example

The shipped example `examples/sample_chat.json`:

```
[
  {"role": "user", "content": "Explain recursion."},
  {"role": "assistant", "content": "Recursion is..."},
  {"role": "assistant", "content": "Recursion is..."}
]
```

Running `contextray optimize examples/sample_chat.json` yields:

```
=== CONTEXTRAY OPTIMIZATION REPORT ===
IMPACT
Original: 48 chars
Optimized: 48 chars
Saved: 0 chars (0%)
TOP WASTE
- Hash Recursion is... repeated 2 times (15 chars wasted)
SAFETY
[OK] System messages preserved
[OK] Cross-role duplicates not removed
[OK] Code blocks protected
[OK] Small chunks skipped
-----
Output saved to: examples/sample_chat_optimized.json
```

The duplicate is 15 chars — below `MIN_CHUNK_SIZE` (64) — so it is detected and reported as waste but deliberately not removed, exactly as the `tiny-chunk` limitation describes. This example is educational: it shows the reporting surface, not a realistic savings scenario.

Realistic scenario (development fixture)

A fixture with repeated payloads, duplicated assistant turns, and code blocks:

```
=== CONTEXTRAY OPTIMIZATION REPORT ===
SEGMENTS
- code: 2 (protected)
- json: 1 (protected)
IMPACT
Original: 3029 chars
Optimized: 1624 chars
Saved: 1405 chars (46%)
TOP WASTE
- Hash 300b1660169c... repeated 2 times (990 chars wasted)
- Hash b8a2b3980551... repeated 2 times (450 chars wasted)
- Hash Sets use has... repeated 2 times (58 chars wasted)
```

11 Appendix B — Quick Reference

Segmentation decisions at a glance

Situation	Segment
```-fenced block with a language tag	code (STRICT)
```-fenced block without a language tag	block (STRICT)
one or more whole lines forming a single parseable JSON value	json (STRICT)
≥ 3 whole lines that parse as valid Python (inputs ≤ 20,000 lines)	code (STRICT)
everything else	text (FLEXIBLE)

Pipeline decisions at a glance

Situation	Result
system role	always KEPT
first occurrence of a key	KEPT — becomes the reference copy
same key again, same role	REMOVED — replaced by marker
same key again, different role	FLAGGED_ONLY — kept, reported as waste
chunk < 64 chars	no hash; dedup by raw text; never replaced
marker would be ≥ chunk length	chunk kept verbatim (no negative reduction)

Constants

```

MAX_CHUNK_SIZE      = 1000    # split ceiling (paragraphs -> lines -> whitespace)
MIN_CHUNK_SIZE      = 64      # below this: hash = None, never removed
CODE_FENCE_REGEX    = ```.*?``` (DOTALL) # masked before splitting
DEDUP_KEY           = hash or raw text for tiny chunks
MARKER              = [duplicate of chunk #N removed] (33 chars)
TOKEN_HEURISTIC     = chars / 4 (or custom token_estimator)
SEGMENT_PLACEHOLDER = __SEG<N>__ + U+FEFF padding (length-preserving)
MAX_LINES_FOR_PYTHON_SCAN = 20000 # unfenced-Python firewall (None = unlimited)
MAX_INPUT_MB (CLI)  = 50.0    # input size guard

```

CLI facts

- Auto-detects input: JSON message list → `optimize_context()`; any other text → `optimize_text()` with a `SEGMENTS` section. `--strict` rejects text inputs.
- Default output name: `<input>_optimized<ext>` — same directory as the input; text inputs always become `..._optimized.json`.
- Exit codes: 0 success / 1 error / 2 usage.
- `--stdout` writes pure JSON to stdout; header and report go to stderr.
- Emoji markers auto-fall back to ASCII on legacy consoles.

Project facts

- Package name `contextray` 0.3.0; MIT license; Python ≥ 3.9; zero runtime dependencies.
- Tests: 18 API + 181 pipeline + 23 segmentation checks, all passing (Section 5).
- Two public entry points: `optimize_context()` (chat) and `optimize_text()` (raw text with structural protection).
- This document was generated from the v0.3 codebase and README; figures reflect observed runs.